ChatGPT

深度扩展：用组装式思维重构 AIMes —— 从“死板系统”到“乐高平台”

一、破题：为什么“组装式”是 MES 产品化的唯一出路?

传统 MES 因“标准化 vs 定制化”而陷入两难：选用现成套件（COTS）常导致“削足适履”、不能满足车间千奇百怪的流程；而完全定制开发虽然贴合业务，却耗时耗资、后期维护成本高且难升级 [1] [2]。这种悖论正是传统 MES 的痛点。针对这一问题，西门子 Mendix 提出的组装式 MES（Composable MES）概念，采用**“核心稳态 + 边缘敏态”**的架构思路 [3]。其中，“核心”由 IT 团队定义标准化的数据模型和核心逻辑（如工单、BOM、设备等），保持稳定一致；“边缘”则允许通过低代码快速配置车间特有的流程、表单和看板等业务逻辑 [3]。正如报道所述，组装式 MES 承认“没有两家工厂是完全一样”的现实， [4] 因此赋予企业 IT 团队自主构建100%贴合自身业务的 MES 能力，而非再把自己套进僵化的单体系统里。西门子中国区总经理也指出，Mendix 低代码平台正是支撑这一转变的基石，它让 MES 不再是昂贵的“黑盒”，而成为企业可驾驭、可演进的数字化资产 [5]。

二、核心架构拆解：核心稳态 + 边缘敏态

组装式架构的核心在于双模运行：**IT 主导的核心稳态层**与**OT（运营）主导的边缘敏态层**。核心稳态层负责整个系统的基础框架和稳定性，包括：

- **数据模型稳态**：核心的生产数据模型（如工单、BOM、工艺路线、设备主数据）固定不变，由 IT 统一管理。
- **核心算法稳态**：AIMes 的灵魂——**AI 排产引擎**，作为输入工单与资源约束输出最优排产计划的核心算力模块，必须保持与特定车间 UI 和设备解耦。
- **权限与隔离**：多租户隔离、角色权限模型等基础安全策略，属于平台级稳定逻辑。

边缘敏态层则关注**车间运营的多样需求**：

- **流程敏态**：各车间的业务流程（如报工流程是否包含质检、返工如何处理等）可以通过可视化建模或配置化编排来定制。
- **表单敏态**：现场操作界面（如扫码报工界面、质量异常记录表单等）可动态配置字段。
- **看板敏态**：针对不同角色（厂长、主任、班长）的自定义看板布局和指标卡片，可通过拖拽组装完成。

综上，组装式架构实质是**“骨架+肌肉”**：骨架（Core）由 IT 打造统一标准，保证系统内核稳定 [6]；肌肉（Extension）由业务侧利用低代码快速延展，满足个性化需求 [6]。这种设计理念使得 AIMes 平台既具备工业级可靠性，又可灵活适配客户的特殊场景。 [3]

三、AIMes 组装式重构落地实战（.NET 10 + Vue 3）

在现有技术栈下，我们不引入昂贵的外部低代码平台，而是用代码自建**微内核+插件**体系，实现“核心稳态+边缘敏态”。核心思路是：保持单体部署、严格模块化，用内部事件总线与插件机制支持可编排的业务流程。

Step 1: 后端 — 从“贫血模型”到 DDD+服务化

- **模块化核心服务**：将原有单体中的关键功能拆分为内部隔离的业务模块，每个模块拥有自己的业务逻辑和数据。比如，把原有的 `SchedulingRuleEngine` 封装为一个独立的排产服务（`SchedulingGrain`），仅对外暴露一个 `GeneratePlan(constraints)` 接口，用于生成最优计划[7]。类似地，将物料齐套、库存、订单管理等核心功能分解为独立领域服务，各自隔离数据，只通过事件或接口交互。这样做符合“模块化单体”理念：**每个模块拥有自己的逻辑和数据，只通过 API 或事件与外部交互**[7]。
- **事件驱动和 Saga 编排**：业务流程不再写死在单个 Service 中，而是通过领域事件解耦和编排。例如，操作工报工后发布 `WorkReported` 事件，触发一系列监听器：子模块 A 更新工单状态，模块 B 扣减库存，模块 C（可选）启动质检流程……如果某一步骤失败，Saga 模式可以通过补偿事件回滚之前步骤，实现全局一致性[8]。如报道所述，Saga 允许每个步骤本地提交事务并发布事件，出错时由补偿逻辑回滚，保证系统最终一致[8]。使用这种模式，车间流程就能灵活可配置：无需改动代码，只需在编排配置中移除或重排某些事件监听，即可零代码满足不同工厂的流程差异。
- **预留业务钩子**：在核心流程中埋点，允许租户加载自定义逻辑。例如，在工单下达前提供一个 `BeforeOrderRelease` 钩子，特定租户可绑定自己的校验（如检查操作人员资质），钩子中逻辑出错则阻止下达。这种机制让边缘业务改动仅靠配置即可实现，提高敏捷度。

Step 2: 前端（Vue 3）— 从页面开发到视图配置化

前端开发方向也是“稳态组件 + 动态布局”。

- **稳态 UI 组件**：将核心功能的视图组件封装为不可拆分的原子组件。例如，基于 `dhtmlx-gantt` 或类似库构建的甘特图组件，专注显示排产结果，支持拖拽调整和冲突检测；物料齐套矩阵组件用于 OEE 分析；这些组件内核封闭，对外只暴露标准接口。
- **敏态 UI 引擎**：所有报工表单、异常录入、看板等通过 JSON Schema 或配置驱动生成。可以使用像 SurveyJS Vue Form Builder 这样的开源组件，动态渲染后端定义的表单结构[9]。例如，当车间需要增加“设备温度”字段，只需在后台上传新的 JSON Schema，前端自动渲染相应输入框并绑定到 `EntityExtension` 扩展表，无需改前端代码[9]。看板界面可采用像 `vue-grid-layout` 的布局库：提供“KPI卡片”、“图表”、“列表”等积木式组件，用户通过拖拽和配置数据源，自行搭建个性化监控大屏。这样，无论前端多么多样化的需求，都可以通过配置而非硬编码来实现。

Step 3: 连接层 — 云边协同的“软网关”

为了实现真正的行业通用和多厂复用，我们必须让设备层差异局限在本地，而核心系统只处理标准化数据。方案是：在工厂部署一个轻量级 Edge Agent（.NET Core 或 Go 实现），专门负责**协议转换和数据统一**。该 Agent 通过 OPC UA、Modbus 等工业协议采集设备和 PLC 数据，将其转换为统一的 JSON/HTTP 接口，并将采集到的数据推送给云端 AIMes 系统；反之，云端下发指令由 Agent 转化为本地可执行命令。Softing 等厂商的实践表明，通过 OPC UA 信息模型标准可以实现 OT 与 IT 的无缝连接[10]：“EdgeConnector” 工具就是为此设计，它能够在控制系统与云之间“提供无缝通信”[10]。我们可以借鉴这一思路：只需在边缘配置好不同设备模型的映射脚本，核心系统即可不动声色地复用相同接口对接任何车间硬件，大大降低了多厂适配难度。

四、组装式重构的商业化变现：AIMes 的“App Store”

组装式思维不仅改变技术架构，也带来全新的商业模式。一旦系统模块化，就可以像“乐高积木”一样进行价值交付：

- **降维交付模式**：传统 MES 项目需要月余甚至年级周期才能上线。组装式交付则大幅压缩时间。引导客户注册后，选择行业模板（包含核心模块的标准化配置），与客户生产人员共同通过拖拽和表单配置在数日内完成核心业务流的搭建。比如一个机加工工厂可加载“机加工行业套件”，立刻获得预置的 B

OM、工艺路线和设备模型，再根据实际需求拖拽组装个性化流程，前端人员只需几天便可把系统从 0 搭建起来，实现快速上线与迭代。

- **AI 排产超级积木**：与其将 AIMes 整包出售，不如将 AI 排产引擎作为一个“即插即用”组件提供。客户可以保持原有 MES，只需调用 AIMes 的 API 传入工单清单，即可获得最优排产方案。这样一来，我们实质上把 AI 排产的核心竞争力拆分出来单独定价，直接对标那些没有 AI 能力的传统 MES 厂商，形成“降维打击”的优势。
- **行业 ISV 生态**：稳态核心提供数据和算法基础，而敏态领域完全开放给生态伙伴。任何行业 ISV（独立软件开发商）都可以在 AIMes 平台上开发特定扩展模块（如 SMT 贴片专用报工插件、AOI 质检规则、特种设备对接器等），并在平台商城出售。AIMes 对成交额进行分成，这样一来，专业 ISV 帮我们解决细分行业需求，我们则省却了培养全能团队的压力。这种模式类似 SaaS 平台上的应用商店，其背后的飞轮效应也很明显：随着使用者增加、数据积累，我们可以提供更精准的行业基准报告和优化建议，进一步增强产品粘性 [11]。

五、实施路径：如何避免在重构中“翻车”？

重构务必**渐进式推进**，采用“绞杀者模式 + 敏捷迭代”来控制风险：

1. **Phase 1（1 个月）：敏态切口**。先从表单和规则配置化入手，不动核心架构。将硬编码的报工表单改为 JSON Schema 驱动，或将少量流程用可配置工具（如状态机或低代码引擎）表现出来。让团队和客户先尝到“零代码定制”的甜头。
2. **Phase 2（2 个月）：稳态剥离**。将 AI 排产引擎从主程序中物理提取出来，作为独立服务运行（可用内部 API 或 gRPC 调用）。同时完善多租户支持（如先用脚本快速创建独立数据库租户 [12]），验证核心与边缘完全解耦的可行性。
3. **Phase 3（3 个月）：编排上台**。在已有核心功能基础上引入事件总线，将级联操作改为事件监听。这样当一个模块变更时，只需要监听相应事件便可触发后续流程，为后续灵活编排奠定基础。
4. **Phase 4（持续）：构建行业模板库**。每服务一个客户，就沉淀出一套领域模板（稳态+敏态配置）。这些模板可以快速应用于新客户，加速后续交付。

整个过程中避免陷入微服务泥潭：**先保留单体部署，只做内部模块化**。如业界所倡“Monolith First”原则 [13] 所示，早期需要快速迭代的项目用单体多模块架构能“更快交付、更低成本”，而微服务的可重用性优势要等到真正遇到性能瓶颈或规模扩展才考虑 [14] [15]。小团队长期维护过度拆分的系统只会增加运营负担 [16]。在每个迭代阶段，都以**客户付费、市场反馈**为导向：首先验证客户是否愿意为 AI 排产买单，再决定下一步架构演进的方向。

总结：落地执行的灵魂三问

每周工作都用以下标准自检：

- **这周的研发，让客户愿意多付钱了吗？**（关注商业价值，一切以解决真实痛点为目的）
- **我们是在打造“另一个黑湖”，还是在做“制造业的 ChatGPT”？**（始终以 AI 排产为核心锚点，基础 MES 功能只做及格线，不瞄准臃肿复杂）
- **5 个人的团队能承受这个架构吗？**（拒绝微服务冲动，坚持模块化单体，才是初创期SaaS的可行之路）

通过以上“敏态配置——稳态剥离——AI算力商业化”的渐进路径，AIMes 能在半年内快速验证市场和架构可行性，逐步建立自己的 AI 制造壁垒。

参考资料： 多篇行业报告和技术博客指出，传统 MES 多采用重开发模式面临长周期和高维护成本 [1] [2]；Mendix 提出的组装式 MES 强调“核心稳态+边缘敏态”，通过低代码组件化来打通 OT/IT 间隙 [3] [5]。对于初创SaaS而言，模块化单体架构优于微服务——它能够快速迭代、降低运维复杂度，并满足小团队规模要求 [14] [15]。此外，使用事件驱动和 Saga 模式可以实现松耦合的流程编排 [8]，而边缘 Agent 和 OPC UA 连接器

的实践表明，工厂数据可在本地标准化后安全输送到云端 [10] 。这些理论和实战经验都为 AIMes 的组装式重构提供了有力指导。

---

[1] [3] [4] [5] [6] 制造业的静默革命：用组装式思维重构 MES，打破“标准品”与“定制化”的终极悖论 | 剑指工控
https://jianzhigongkong.com/node/3107

[2] Rethinking MES for Digital Transformation
https://www.machinemetrics.com/blog/rethinking-mes-for-digital-transformation

[7] [8] [13] [15] Architecting Robust .NET Solutions: Modular Monolithic, Clean Architecture, Event-Driven Architecture (EDA), CQRS Guided by Domain-Driven Design | by Mosharraf Hossain | Medium
https://medium.com/@mail2mhossain/architecting-robust-net-dfa4f3725142

[9] Vue Form Builder | Generate dynamic JSON forms in your Vue.js application
https://surveyjs.io/survey-creator/documentation/get-started-vue

[10] Softing Industrial Integrates OPC UA Information Modeling into edge Product Range and Secure Integration Server – OPC Connect
https://opcconnect.opcfoundation.org/2024/12/softing-industrial-integrates-opc-ua-information-modeling-into-edge-product-range-and-secure-integration-server/

[11] Data Network Effects in SaaS Enabled Marketplaces | Tomasz Tunguz
https://tomtunguz.com/saas-enabled-marketplace-data-advantage/

[12] Multi-tenancy - EF Core | Microsoft Learn
https://learn.microsoft.com/en-us/ef/core/miscellaneous/multitenancy

[14] [16] Why Startups Are Replacing Microservices with Unified Full-Stack Architectures | by ISyncEvolution Pvt Ltd | Apr, 2026 | Medium
https://medium.com/@teamisyncevolution/why-startups-are-replacing-microservices-with-unified-full-stack-architectures-c5478b93050b